

SentinelAPI: A Zero-Trust API Vulnerability Scanner via Cross-Principal Differential Testing

Team SentinelAPI

AMIHACKS 2026 — Track C (Industry / Deep-Tech)

Jaipur, Rajasthan, India

Abstract—Broken object-level authorization (BOLA/IDOR) and excessive data exposure are the two most damaging classes of API vulnerability, yet most engineering teams review API security manually, infrequently, and only after deployment. We present SentinelAPI, an automated scanner that detects five vulnerability classes—BOLA, excessive data exposure, broken authentication, broken function-level authorization, and mass assignment—from an OpenAPI specification alone. The core of the approach is *differential testing across security contexts*: the scanner registers two synthetic principals and systematically injects one principal’s identifiers into the other’s session, flagging any endpoint that returns cross-principal data. Independently, it diffs the actual response against the schema declared in the specification to detect over-exposure of sensitive fields, and it compares declared security requirements against observed enforcement. The engine is implemented twice, in Go and Python, behind a common command-line boundary, and is verified against a corpus of two deliberately vulnerable sandbox APIs with precision = recall = F1 = 1.0 (24/24 findings, zero false positives, zero false negatives).

Index Terms—API security, broken object-level authorization, differential testing, OpenAPI, vulnerability scanner

I. INTRODUCTION

Modern applications are assembled from dozens to hundreds of internal and third-party APIs. Breaches that exploit broken object-level authorization (“API1:2023”) and excessive data exposure are a leading cause of production data leakage [1]. The defect is subtle: a request can be syntactically valid and authenticated yet still return another user’s data, because the flaw lives in *authorization logic*, not request formatting. Signature-based web scanners miss this class entirely, enterprise API-security platforms are inaccessible to smaller teams, and manual penetration testing does not scale with deployment velocity.

SentinelAPI closes this gap with a lightweight, deterministic scanner whose correctness is verifiable by construction: it encodes the API’s *security contract* as a set of oracles and tests the contract against observed behavior, rendering explainable, severity-ranked findings with copy-paste reproduction.

II. RELATED WORK

Differential testing [2] runs the same input through two or more contexts and treats divergence as a defect. SentinelAPI applies this to security: the same endpoint is executed under two principals, and any difference in data ownership is a defect.

RESTler [3] generates stateful sequences of API calls from a Swagger/OpenAPI spec to find reliability bugs, using a

dynamically inferred dependency graph. *EvoMaster* [4] applies evolutionary search to REST API test generation. *Morest* [5] adds model-based state fuzzing. These tools target reliability and coverage; our goal is narrower and our oracle is stronger—we test a specific, well-defined authorization property rather than search for crashes.

III. THREAT MODEL AND METHOD

A. Threat model

We assume an attacker who is an *authenticated but unauthorized* user: they possess a valid token but should not be able to read or modify other principals’ objects. This matches the most common real-world API breach profile.

B. Cross-principal differential oracle (BOLA)

The scanner registers two synthetic principals A and B . Registration returns each principal’s identifying fields, inferred generically from the response rather than hardcoded. For every endpoint e with a path parameter p , the scanner substitutes each of B ’s identifiers v into p while presenting A ’s token, and inspects the response for B -owned data:

$$\text{leak}(e, v) = \text{HTTP 200}(R(A, e[p:=v])) \wedge \exists m \in M_B : m \in R(A, e[p:=v]) \quad (1)$$

where $R(A, x)$ is the response to request x under A ’s session and M_B is the set of B ’s identifying values. A satisfied predicate flags a CRITICAL finding. The oracle inspects *content*, not status code alone—a 404 is safe behavior, whereas a 200 containing foreign data is a leak.

C. Schema-contract exposure oracle

The specification declares, for each endpoint, a response schema—the set of fields the server *promises* to return. The scanner computes the set difference between the paths actually returned and the paths declared, recursing into nested objects:

$$\text{extra}(e) = \text{paths}(R(A, e[p:=\text{id}_A])) \setminus \text{schemaPaths}(e) \quad (2)$$

Any undeclared path whose leaf is sensitive (password, ssn, card, token, ...) is flagged HIGH; others MEDIUM.

D. Broken authentication oracle

When the specification declares an API-key security requirement for an endpoint, the scanner probes the endpoint with *no* Authorization header. If the endpoint returns data (HTTP 200), the declared security is not enforced—a broken-authentication finding.

E. Broken function-level authorization (BFLA)

For endpoints under an administrative path prefix, the scanner probes with a regular (non-admin) token. A 200 response indicates a missing role check.

F. Mass assignment

For write endpoints (POST/PUT/PATCH), the scanner injects privileged fields (`role`, `is_admin`) into the request body and flags any endpoint that accepts and reflects them.

IV. SYSTEM ARCHITECTURE

The pipeline is: *parse OpenAPI* $\rightarrow$ *register principals* $\rightarrow$ *generate tests* $\rightarrow$ *execute across security contexts* $\rightarrow$ *oracle evaluation* $\rightarrow$ *severity-ranked report*. The engine is isolated behind a command-line boundary (`--base` $\rightarrow$ `findings.json` + `report.html`), so orchestration is language-agnostic. The product wraps the engine in a FastAPI service exposing a consent-gated, SSRF-guarded scan API and a web dashboard.

V. IMPLEMENTATION

Two engines implement identical logic: a **Go** engine (stdlib-only, single static binary) and a **Python** reference (httpx). Identifier inference is generic—any scalar field returned by the registration endpoint becomes a candidate identifier—so the scanner is target-agnostic. Every finding carries severity, endpoint, a plain-language explanation, a copy-paste `curl` reproduction, and a JSON evidence diff.

VI. EVALUATION

We evaluated against a corpus of two deliberately vulnerable sandbox APIs (a “Bank” API and a “Library” API with different identifier names and flaw mixes), each including secure control endpoints that must produce zero findings.

TABLE I
BENCHMARK RESULTS (BOTH ENGINES)

Metric	Bank	Library
Vulnerability classes	5	3
Seeded findings	19	5
Detected	19	5
False positives	0	0
False negatives	0	0

Aggregated over both targets, the scanner achieves **precision = recall = F1 = 1.0** (24/24 true positives). The secure control endpoints (ownership-enforced reads, admin-gated stats) produced zero findings, confirming absence of false positives on correctly implemented endpoints.

VII. NOVEL CONTRIBUTIONS

- **A spec-derived authorization oracle**—the security contract is encoded as a differential test, not guessed by fuzzing.
- **Generic principal-identifier inference**—no hardcoded field names, enabling the scanner to generalize across API designs.

- **Recursive schema-contract diffing**—detects nested over-exposure, not just top-level field leaks.
- **Declared-vs-enforced security comparison**—detects broken authentication from the specification’s own security declarations.
- **Explainability by construction**—every finding ships a reproduction and evidence diff, yielding zero false-positive noise.
- **A two-target benchmark harness** reporting precision/recall/F1 against seeded ground truth.

VIII. LIMITATIONS AND FUTURE WORK

The scanner currently models single-principal per-endpoint access; role lattices, session (shadow-token) handling, and side-channel timing remain future work, as do LLM-assisted oracles for business-logic leaks and CI/CD integration.

IX. CONCLUSION

SentinelAPI demonstrates that the most damaging API vulnerability classes can be detected deterministically, with high precision and full explainability, by treating API security as a differential-testing problem against the API’s own specification. The working system detects five vulnerability classes with F1 = 1.0 across a two-target benchmark, with a clear path to a Rust-class high-concurrency engine.

REFERENCES

- [1] OWASP Foundation. *OWASP API Security Top 10:2023*. <https://owasp.org/API-Security/>
- [2] W. M. McKeeman, “Differential Testing for Software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [3] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API Fuzzing,” in *Proc. IEEE/ACM ICSE*, 2019.
- [4] A. Arcuri, “RESTful API Automated Test Case Generation with EvoMaster,” *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 1, 2019.
- [5] Y. Liu *et al.*, “Morest: Model-based RESTful API Testing with Execution Feedback,” in *Proc. ISSSTA*, 2022.